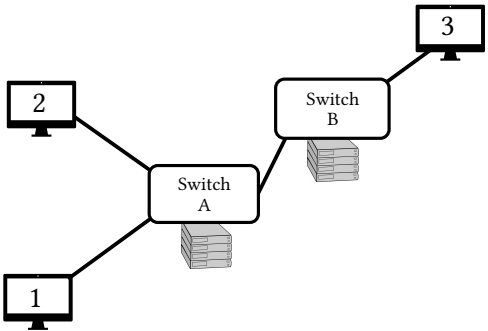
Automata Learning for Network Verification

Mark Moeller

Holy Cross

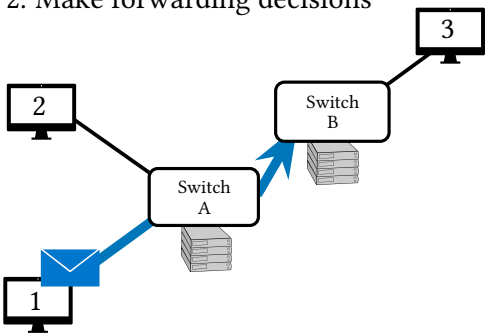
WPI CS Colloquium
Friday, November 13, 2026

Network Routing Policies are Programs



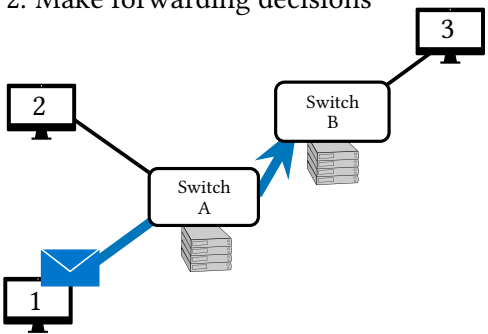
Network Routing Policies are Programs

Forwarding policies:
1. Filter/update packet fields
2. Make forwarding decisions



Network Routing Policies are Programs

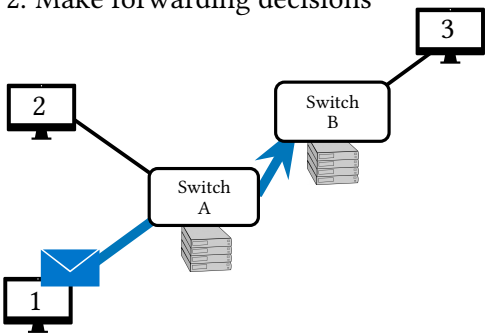
Forwarding policies:
1. Filter/update packet fields
2. Make forwarding decisions



Verification question, e.g.:

Network Routing Policies are Programs

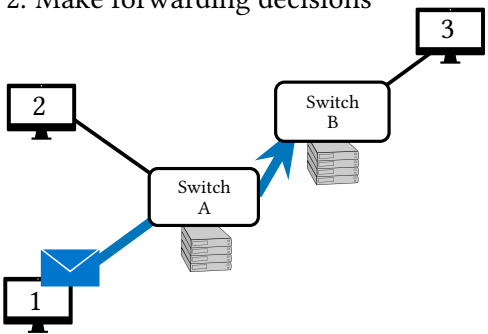
Forwarding policies:
1. Filter/update packet fields
2. Make forwarding decisions



Verification question, e.g.:
Are all hosts reachable from every other host?

Network Routing Policies are Programs

Forwarding policies:
1. Filter/update packet fields
2. Make forwarding decisions

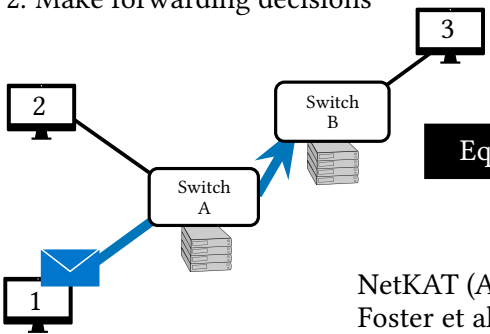


Verification question, e.g.:
Are all hosts reachable from every other host?



Network Routing Policies are Programs

Forwarding policies:
1. Filter/update packet fields
2. Make forwarding decisions



Verification question, e.g.:
Are all hosts reachable from every other host?

Verification query

Equivalence/Adherence



NetKAT (Anderson et al POPL14,
Foster et al POPL15, Moeller/Jacobs et al PLDI24)

Network Verification with NetKAT (POPL 2014)

$$p, q ::= \bot \mid \top \mid f{=}v \mid f{\neq}v \mid f{\leftarrow}v \mid \mathrm{dup} \mid p + q \mid p \cdot q \mid p^{\star}$$

Network Verification with NetKAT (POPL 2014)

Filter packets

$$p, q ::= \bot \mid \top \mid f{=}v \mid f{\neq}v \mid f{\leftarrow}v \mid \mathrm{dup} \mid p + q \mid p \cdot q \mid p^{\star}$$

Network Verification with NetKAT (POPL 2014)

Filter packets

$$p, q ::= \bot \mid \top \mid f{=}v \mid f{\neq}v \mid f{\leftarrow}v \mid \mathrm{dup} \mid p + q \mid p \cdot q \mid p^{\star}$$

Modify packets

Network Verification with NetKAT (POPL 2014)

Filter packets

$$p, q ::= \bot \mid \top \mid f{=}v \mid f{\neq}v \mid f{\leftarrow}v \mid \text{dup} \mid p + q \mid p \cdot q \mid p^\star$$

Modify packets

- NetKAT is sound, complete, and decidable
- Program equivalence is *automata equivalence*

Network Verification with NetKAT (POPL 2014)

Filter packets

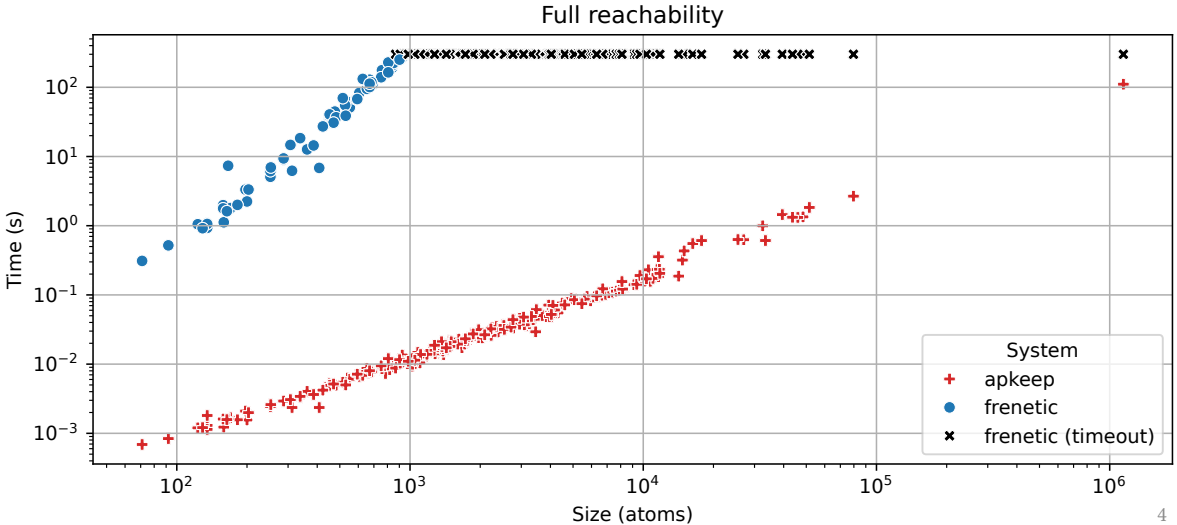
$$p, q ::= \bot \mid \top \mid f{=}v \mid f{\neq}v \mid f{\leftarrow}v \mid \text{dup} \mid p + q \mid p \cdot q \mid p^\star$$

Modify packets

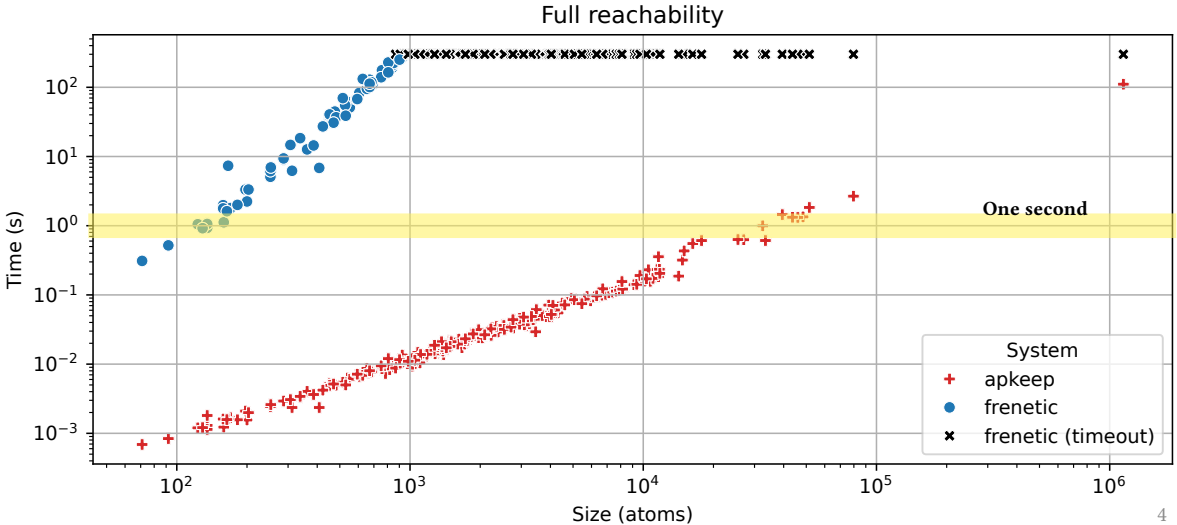
- NetKAT is sound, complete, and decidable
- Program equivalence is *automata equivalence*

Naive approach: Convert program and spec. to Deterministic Finite Automata (DFAs)

Problem #1: NetKAT • is slower than State-of-Art +

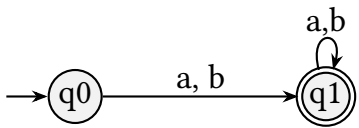


Problem #1: NetKAT • is slower than State-of-Art +



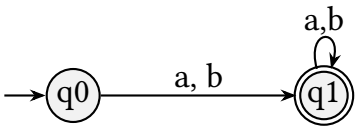
Key Challenge #1:
Huge packet space means huge *alphabet*

Naively in DFAs, transitions are labeled by alphabet symbols

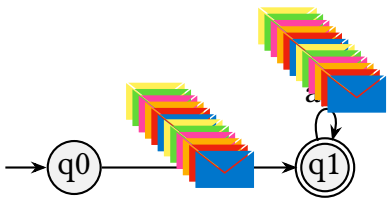


Key Challenge #1:
Huge packet space means huge *alphabet*

Naively in DFAs, transitions are labeled by alphabet symbols

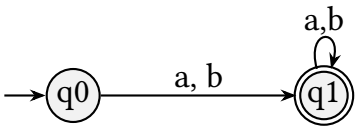


But the packet space is exponential in the header length

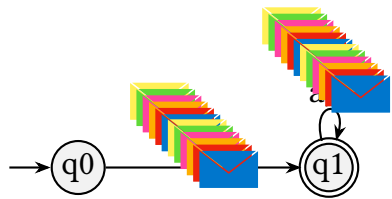


Key Challenge #1:
Huge packet space means huge *alphabet*

Naively in DFAs, transitions are labeled by alphabet symbols



But the packet space is exponential in the header length



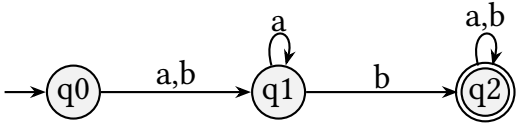
Insight: represent sets of packets symbolically

Key Challenge #2:
We want to model *update* behavior

In DFAs, transitions are taken
independent of previous symbols

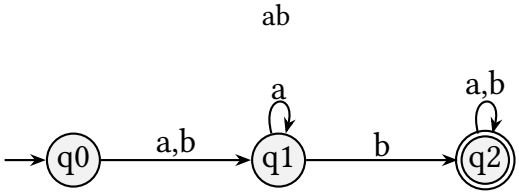
Key Challenge #2:
We want to model *update* behavior

In DFAs, transitions are taken independent of previous symbols



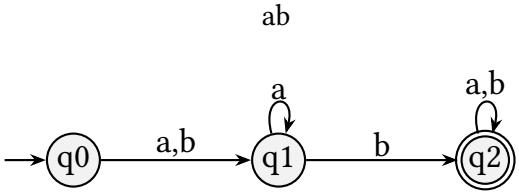
Key Challenge #2:
We want to model *update* behavior

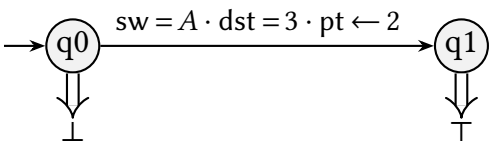
In DFAs, transitions are taken independent of previous symbols



Key Challenge #2:
We want to model *update* behavior

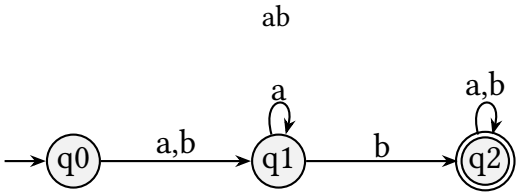
In DFAs, transitions are taken independent of previous symbols

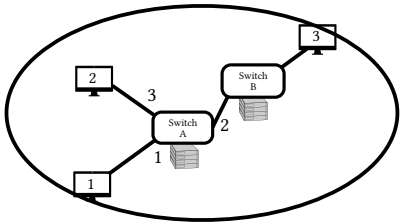


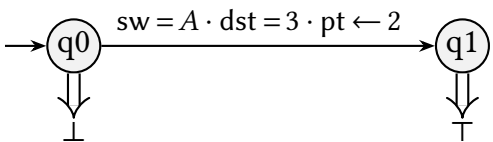


Key Challenge #2:
We want to model *update* behavior

In DFAs, transitions are taken independent of previous symbols

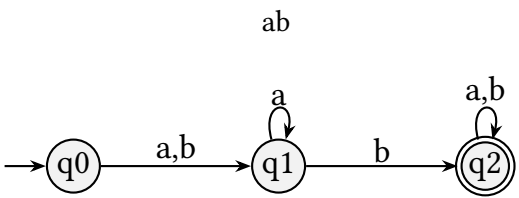


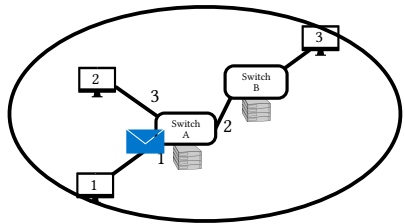




Key Challenge #2:
We want to model *update* behavior

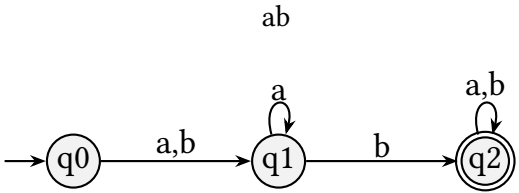
In DFAs, transitions are taken independent of previous symbols

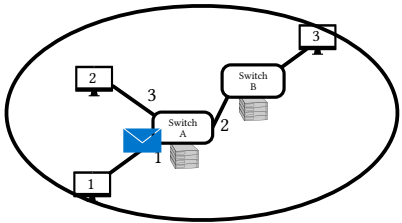




Key Challenge #2:
We want to model *update* behavior

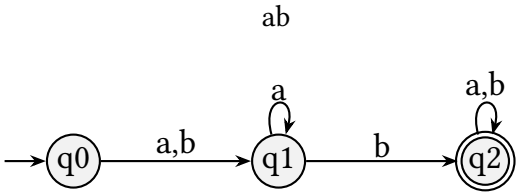
In DFAs, transitions are taken independent of previous symbols

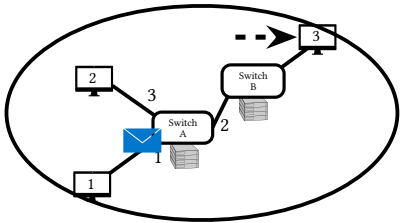




Key Challenge #2:
We want to model *update* behavior

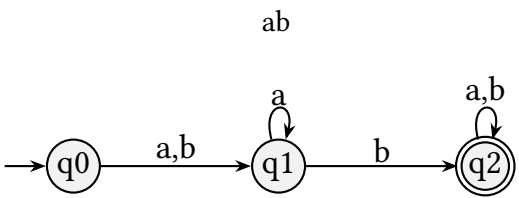
In DFAs, transitions are taken independent of previous symbols

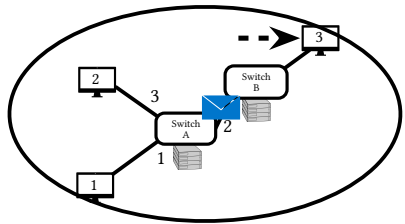


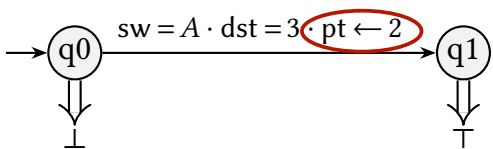


Key Challenge #2:
We want to model *update* behavior

In DFAs, transitions are taken independent of previous symbols

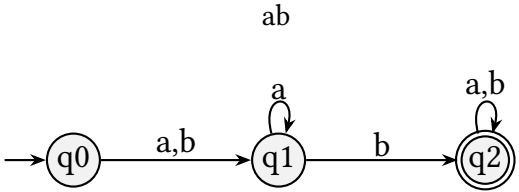


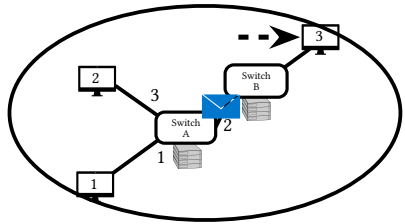




Key Challenge #2:
We want to model *update* behavior

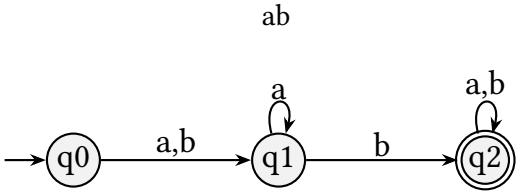
In DFAs, transitions are taken independent of previous symbols

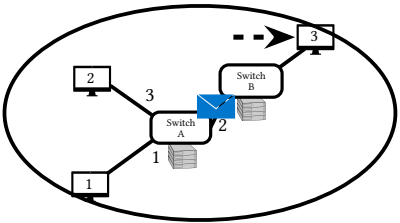




Key Challenge #2:
We want to model *update* behavior

In DFAs, transitions are taken independent of previous symbols

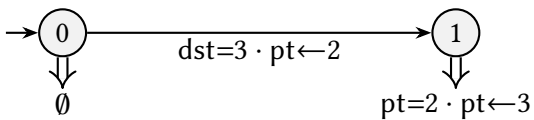




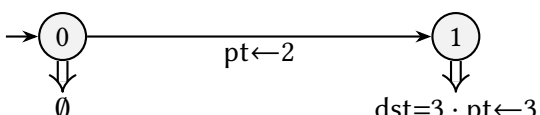
What's hard about NetKAT equivalence?

Are these two NetKAT automata equivalent?

Policy

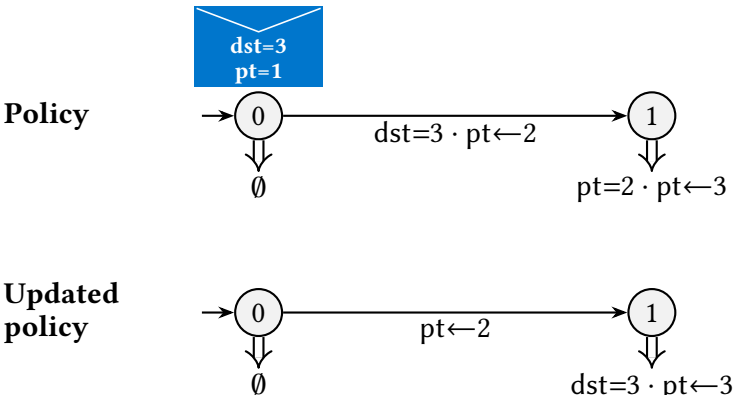


Updated policy



What's hard about NetKAT equivalence?

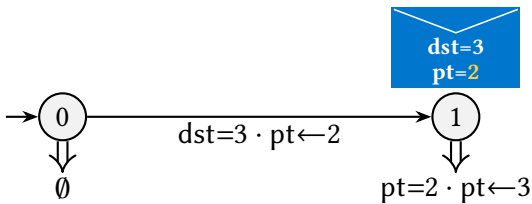
Are these two NetKAT automata equivalent?



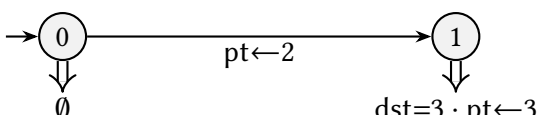
What's hard about NetKAT equivalence?

Are these two NetKAT automata equivalent?

Policy



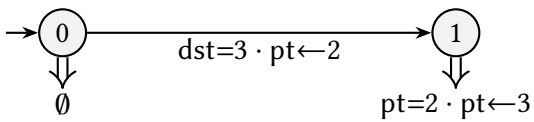
Updated policy



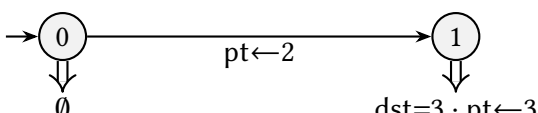
What's hard about NetKAT equivalence?

Are these two NetKAT automata equivalent?

Policy

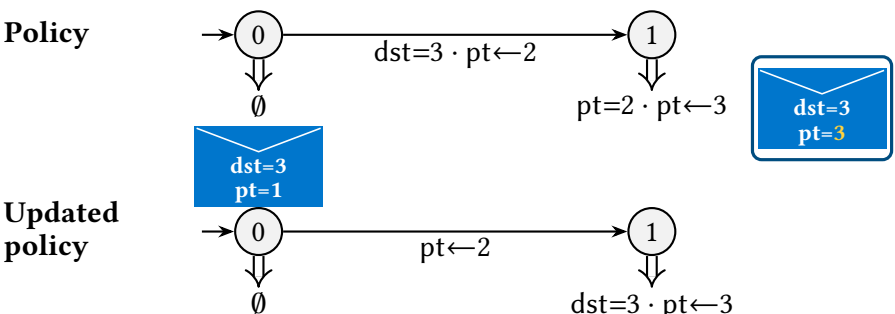


Updated policy



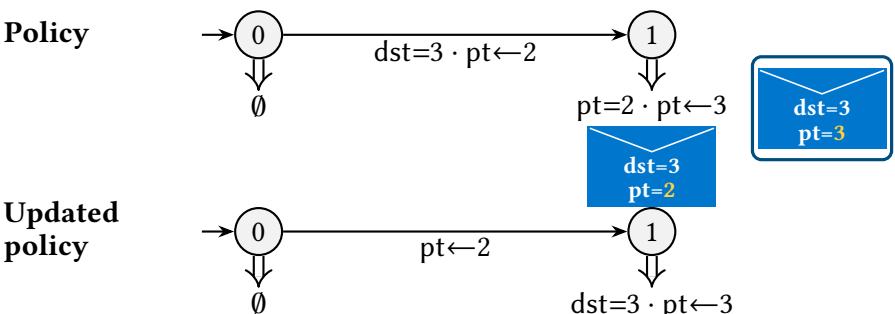
What's hard about NetKAT equivalence?

Are these two NetKAT automata equivalent?



What's hard about NetKAT equivalence?

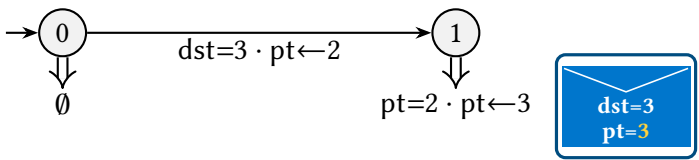
Are these two NetKAT automata equivalent?



What's hard about NetKAT equivalence?

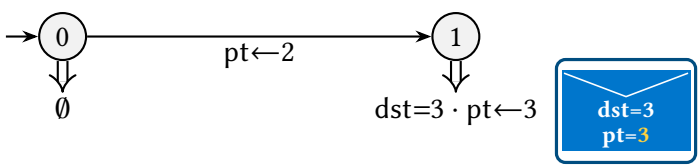
Are these two NetKAT automata equivalent?

Policy



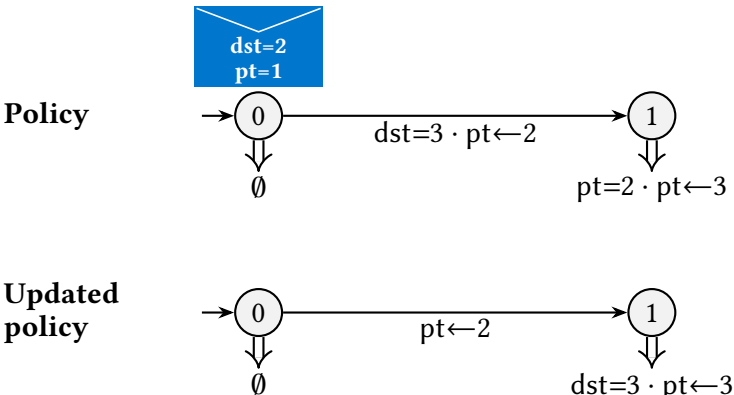
same ✓

Updated policy



What's hard about NetKAT equivalence?

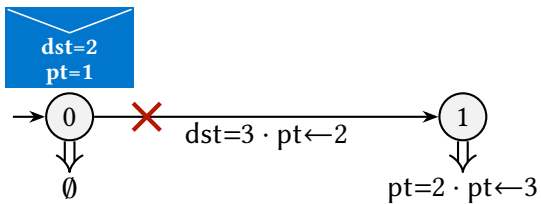
Are these two NetKAT automata equivalent?



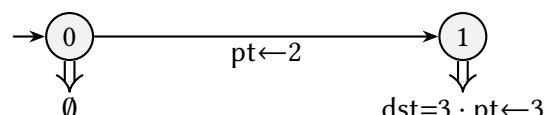
What's hard about NetKAT equivalence?

Are these two NetKAT automata equivalent?

Policy

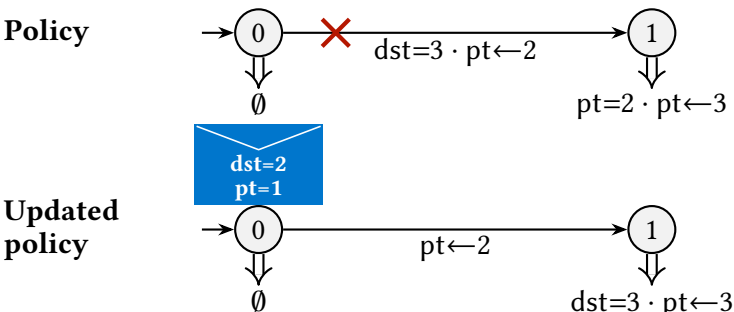


Updated policy



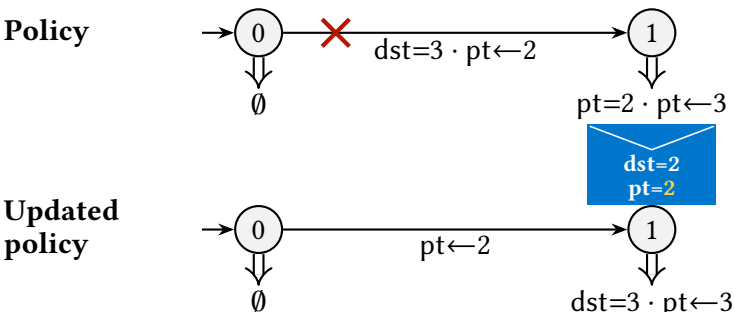
What's hard about NetKAT equivalence?

Are these two NetKAT automata equivalent?



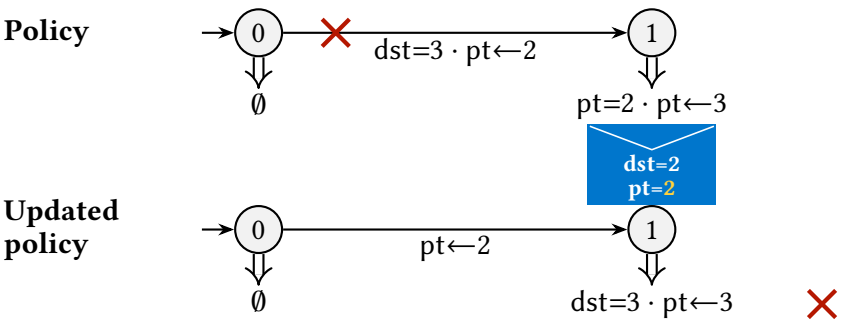
What's hard about NetKAT equivalence?

Are these two NetKAT automata equivalent?



What's hard about NetKAT equivalence?

Are these two NetKAT automata equivalent?

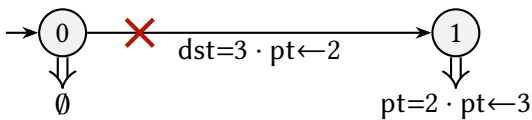


same ✓

What's hard about NetKAT equivalence?

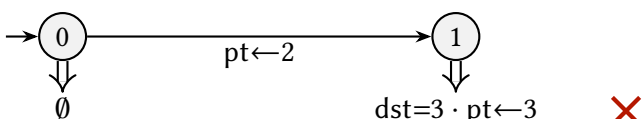
Are these two NetKAT automata equivalent?

Policy



same ✓

Updated policy



Naively we'd have to run every packet! Only 2^{100} to go…

Problem #1: NetKAT verification is slow

Solution: KATch (Moeller/Jacobs et al, PLDI 2024)

Problem #1: NetKAT verification is slow

Solution: KATch (Moeller/Jacobs et al, PLDI 2024)

Challenge #1:
huge packet space, huge alphabet

$\Rightarrow$

Symbolic packets:
sets of packets as BDDs

Problem #1: NetKAT verification is slow

Solution: KATch (Moeller/Jacobs et al, PLDI 2024)

Challenge #1:
huge packet space, huge alphabet

$\Rightarrow$

Symbolic packets:
sets of packets as BDDs

Challenge #2:
transitions update packets

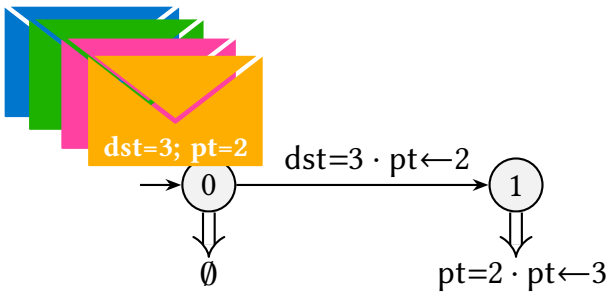
$\Rightarrow$

Symbolic packet programs (SPPs):
transitions as filters + assignments

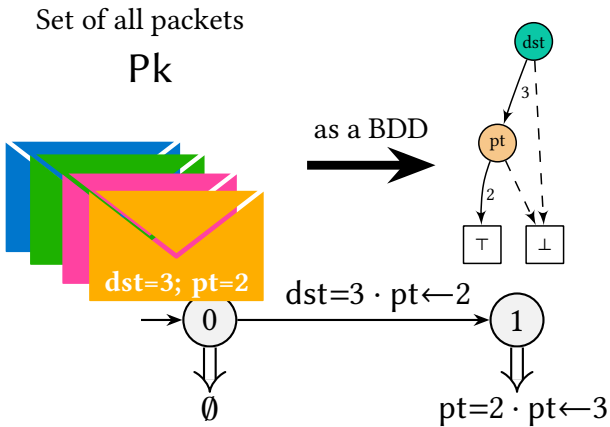
Represent Sets of Packets Symbolically

Set of all packets

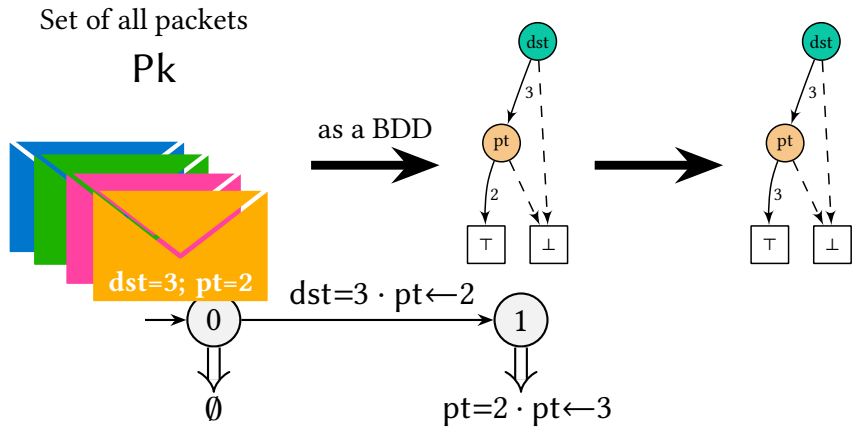
Pk



Represent Sets of Packets Symbolically

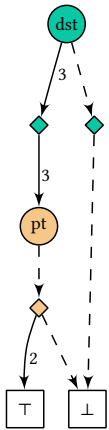


Represent Sets of Packets Symbolically



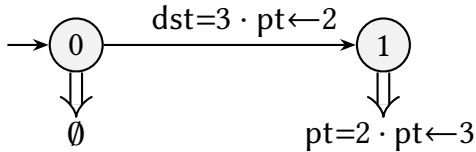
Represent Transitions Symbolically

Symbolic Packet Program (SPP)



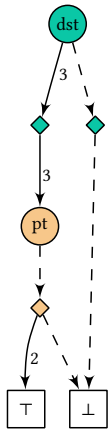
Circle-layers are filters

Diamond-layers are assignments



Represent Transitions Symbolically

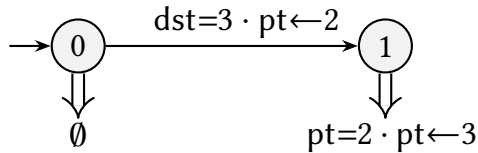
Symbolic Packet Program (SPP)



Circle-layers are filters

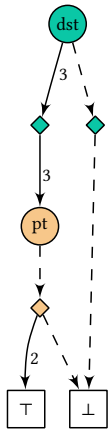
Diamond-layers are assignments

SPPs are canonical: same behaviour ⇒ same SPP, so checking equivalence is a structural comparison, and hash-consing is a big win.



Represent Transitions Symbolically

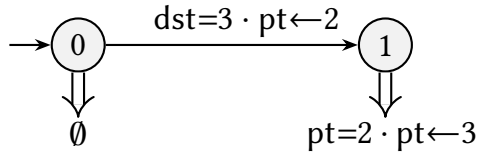
Symbolic Packet Program (SPP)



Circle-layers are filters

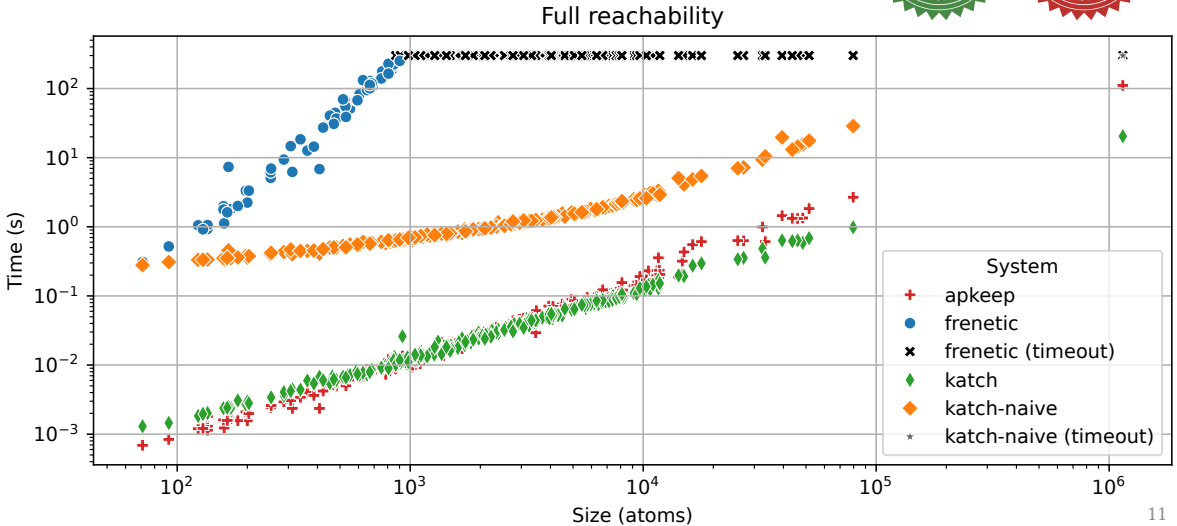
Diamond-layers are assignments

SPPs are canonical: same behaviour ⇒ same SPP, so checking equivalence is a structural comparison, and hash-consing is a big win.

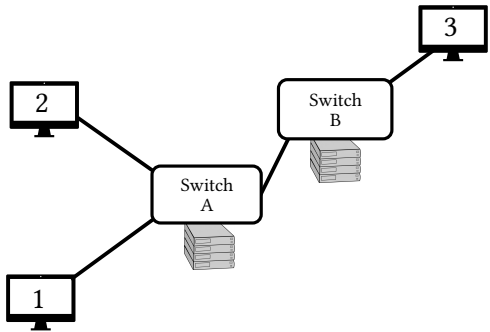


SPPs and symbolic automata are closed under $+, \cdot, \star, \cap, -, \oplus$; $p \equiv q \iff p \oplus q \equiv \bot$

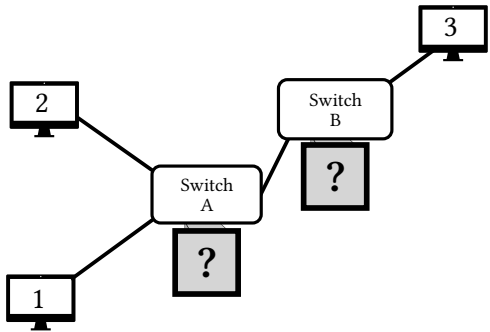
KATch vs. APKeep: Competitive!



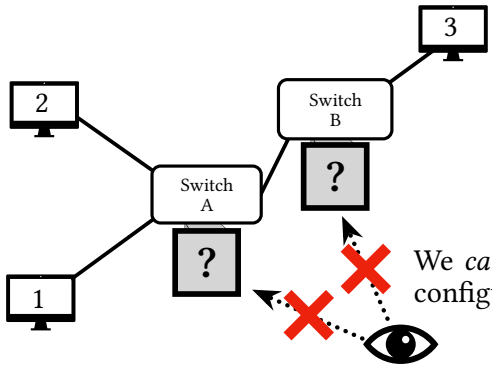
Problem #2: What if we have unknowns?



Problem #2: What if we have unknowns?



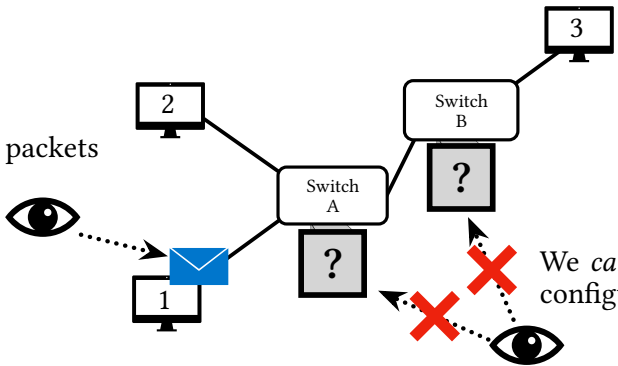
Problem #2: What if we have unknowns?



We *cannot* look at the switch configurations/policies

Problem #2: What if we have unknowns?

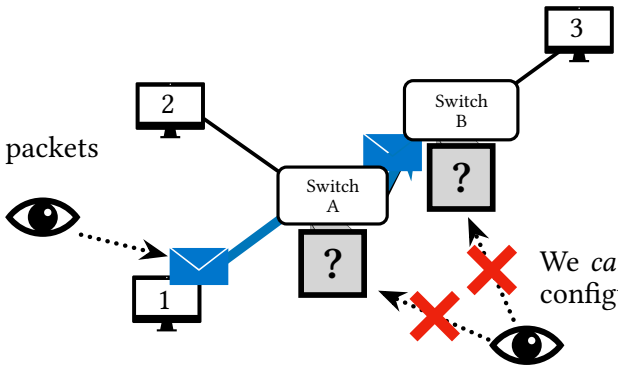
We *can* send packets



We *cannot* look at the switch configurations/policies

Problem #2: What if we have unknowns?

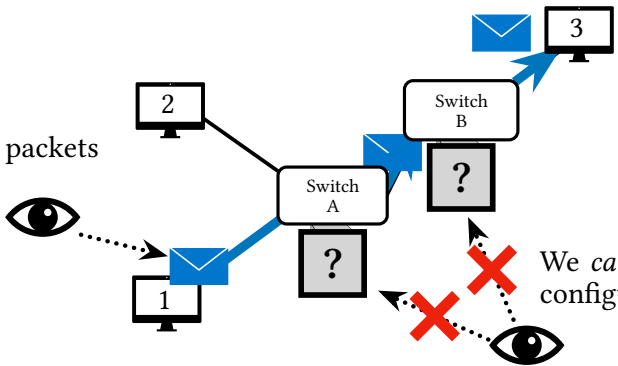
We *can* send packets



We *cannot* look at the switch configurations/policies

Problem #2: What if we have unknowns?

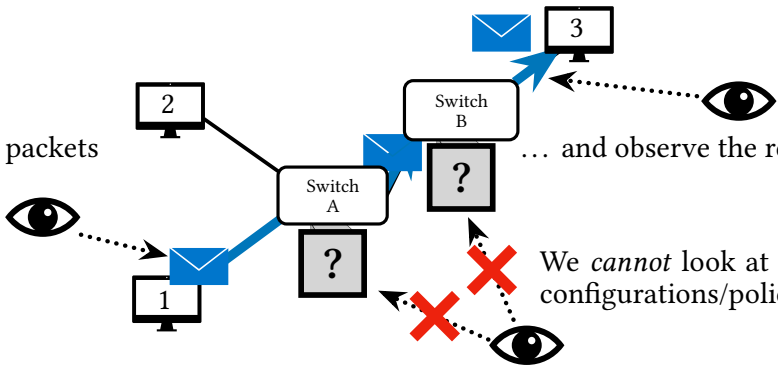
We *can* send packets



We *cannot* look at the switch configurations/policies

Problem #2: What if we have unknowns?

We *can* send packets

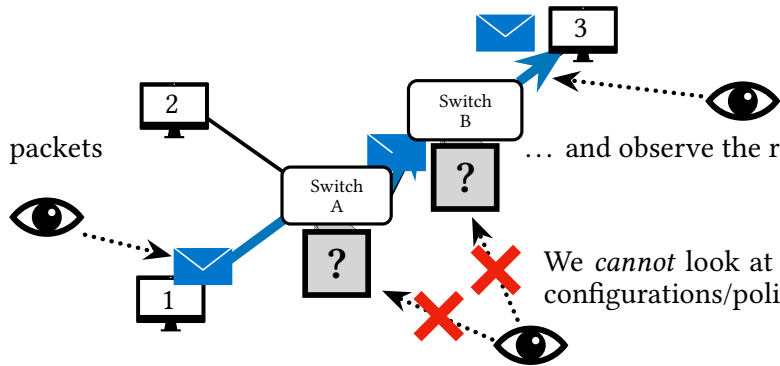


… and observe the resulting trace

We *cannot* look at the switch configurations/policies

Problem #2: What if we have unknowns?

We *can* send packets



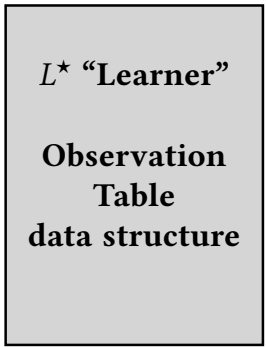
… and observe the resulting trace

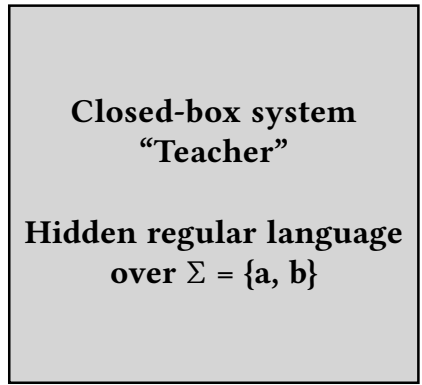
We *cannot* look at the switch configurations/policies

Can we build a model of the network by observing traces?

Background: Active Learning of Finite Automata

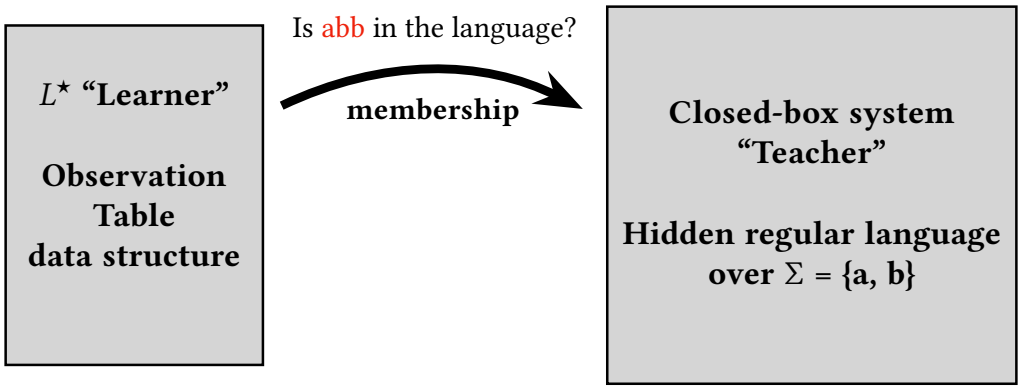
$L^\star$ algorithm learns DFAs from "Minimally Adequate Teacher" (Angluin, 1987)





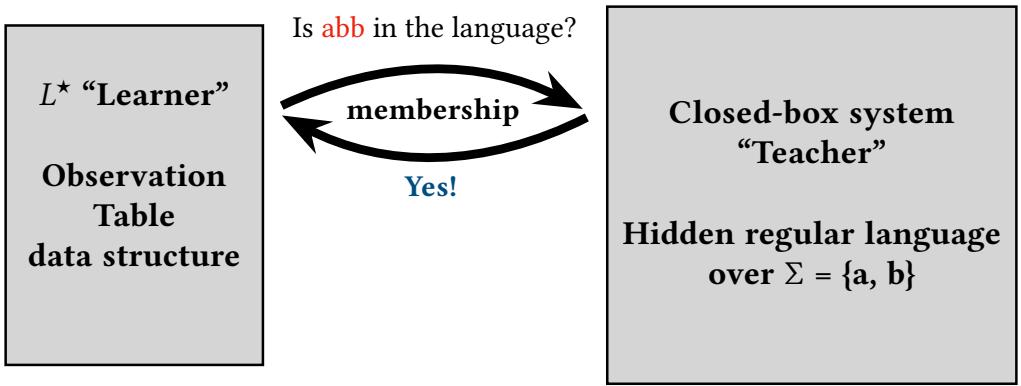
Background: Active Learning of Finite Automata

$L^\star$ algorithm learns DFAs from "Minimally Adequate Teacher" (Angluin, 1987)



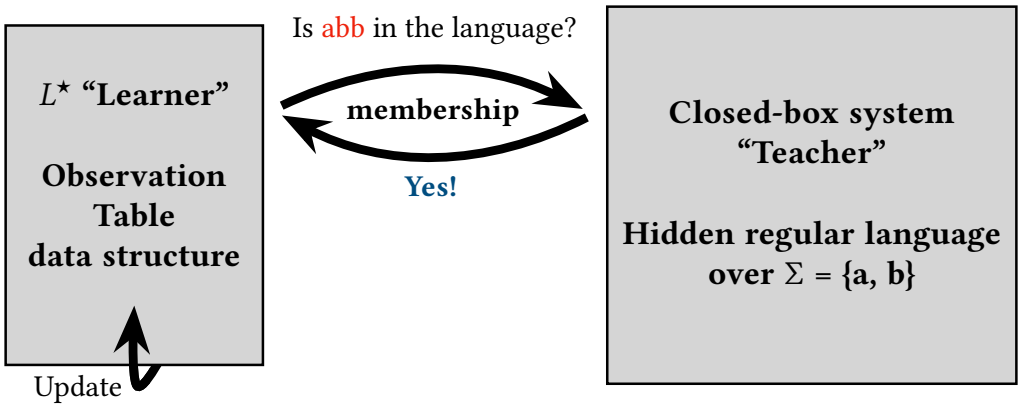
Background: Active Learning of Finite Automata

$L^\star$ algorithm learns DFAs from "Minimally Adequate Teacher" (Angluin, 1987)



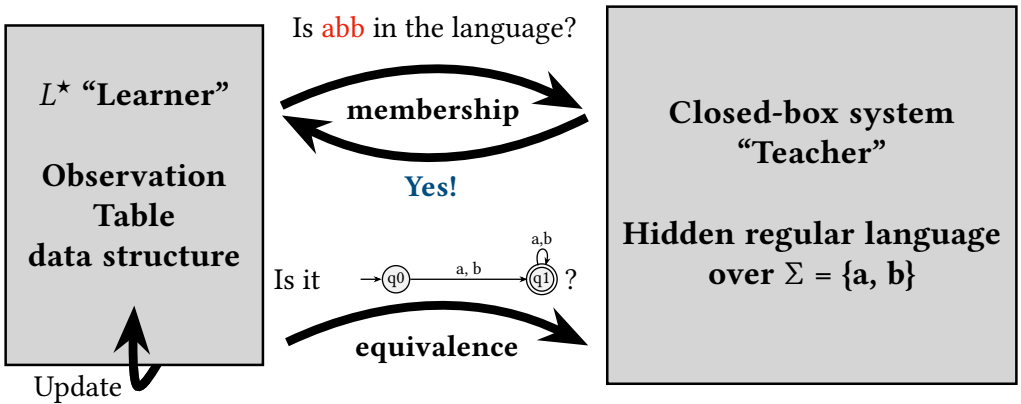
Background: Active Learning of Finite Automata

$L^\star$ algorithm learns DFAs from "Minimally Adequate Teacher" (Angluin, 1987)



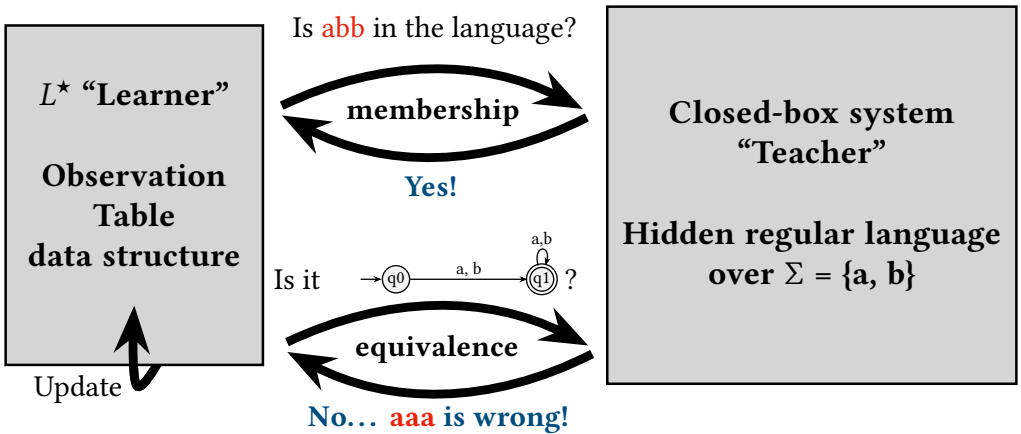
Background: Active Learning of Finite Automata

$L^\star$ algorithm learns DFAs from "Minimally Adequate Teacher" (Angluin, 1987)



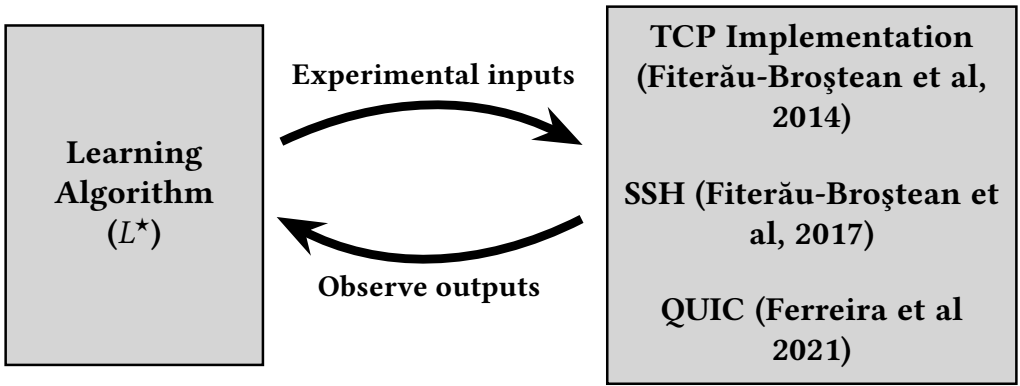
Background: Active Learning of Finite Automata

$L^\star$ algorithm learns DFAs from "Minimally Adequate Teacher" (Angluin, 1987)

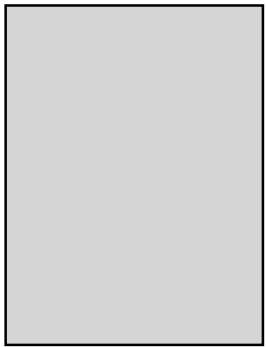


Background: Active Learning of Finite Automata

$L^\star$ has had success in networking already!



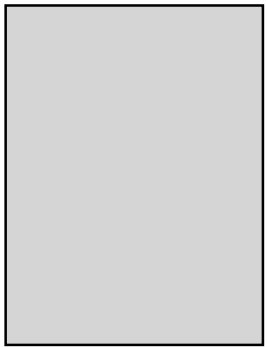
Research Question:
Can we build a model of the network by observing traces?





Research Question:
Can we build a model of the network by observing traces?

Proposed Solution: Adapt $L^\star$ to NetKAT

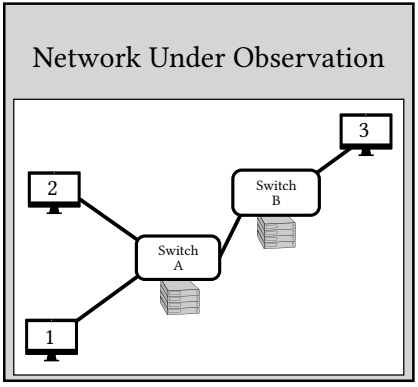




Research Question: Can we build a model of the network by observing traces?

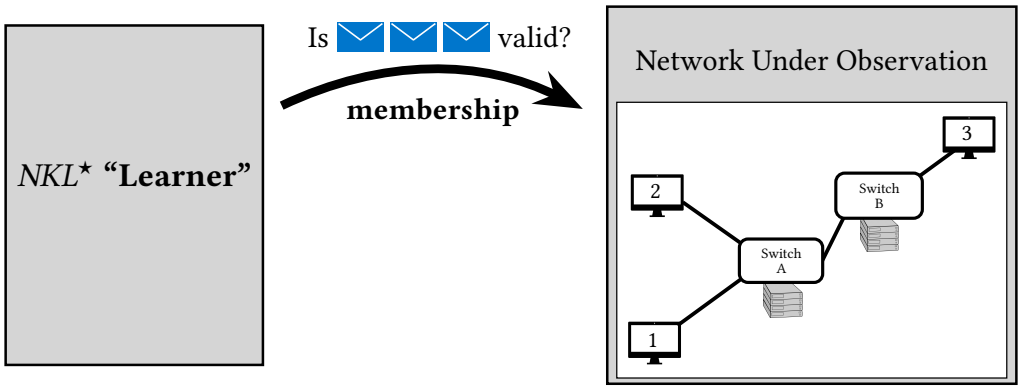
Proposed Solution: Adapt $L^\star$ to NetKAT

NKL* **"Learner"**



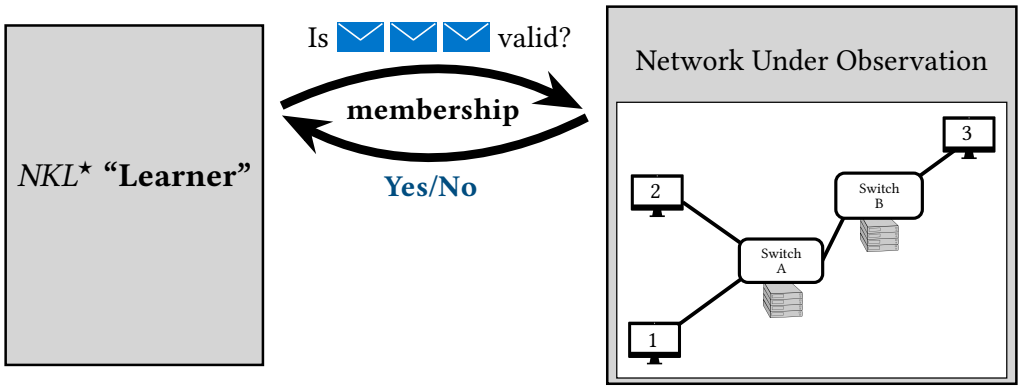
Research Question:
Can we build a model of the network by observing traces?

Proposed Solution: Adapt $L^\star$ to NetKAT



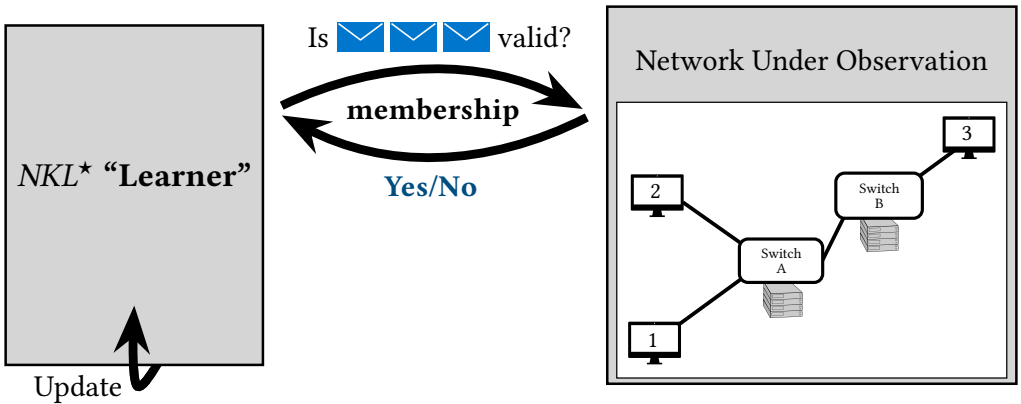
Research Question: Can we build a model of the network by observing traces?

Proposed Solution: Adapt $L^\star$ to NetKAT



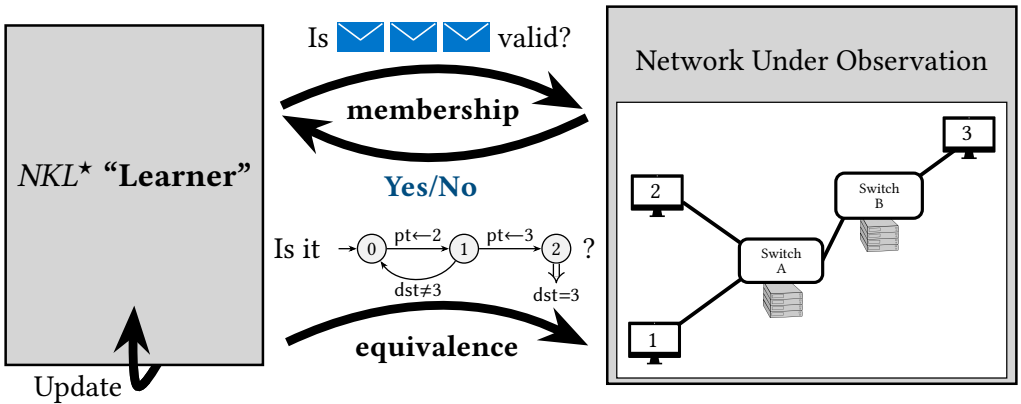
Research Question: Can we build a model of the network by observing traces?

Proposed Solution: Adapt $L^\star$ to NetKAT



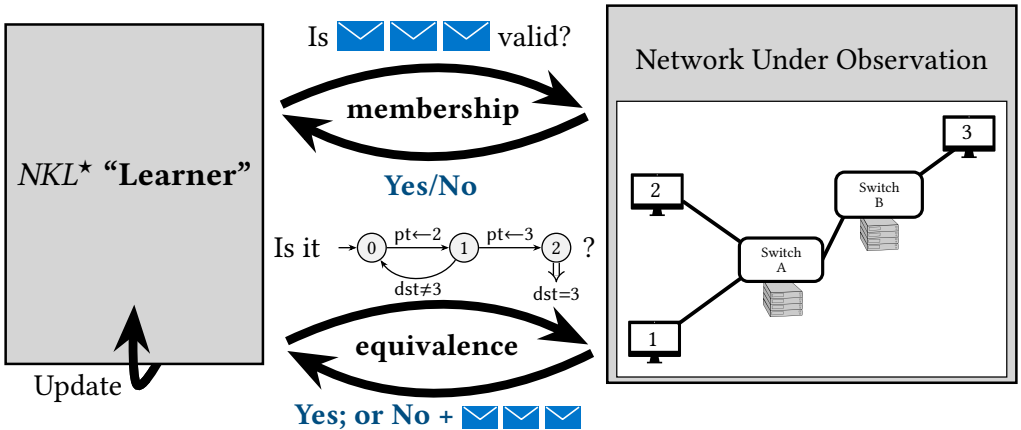
Research Question: Can we build a model of the network by observing traces?

Proposed Solution: Adapt $L^\star$ to NetKAT



Research Question: Can we build a model of the network by observing traces?

Proposed Solution: Adapt $L^\star$ to NetKAT



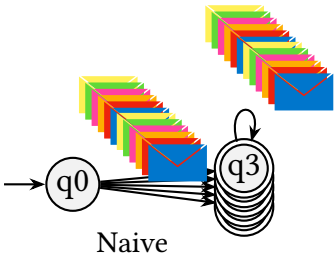
Challenge #1:
huge packet space, huge alphabet

Challenge #2:
transitions update packets

Challenge #1:
huge packet space, huge alphabet

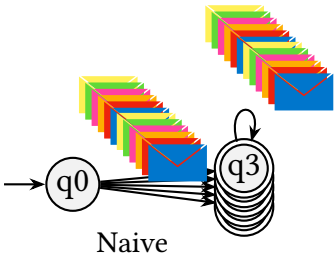
Challenge #2:
transitions update packets

Naive answer: Just build DFAs
(accept the state space blowup!)



Naive

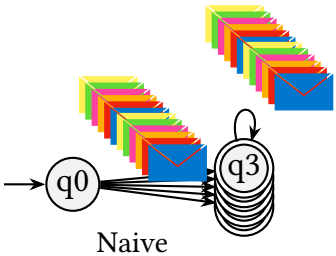
Naive answer: Just build DFAs
(accept the state space blowup!)



Naive

Naive answer: Just build DFAs (accept the state space blowup!)

Better answer:

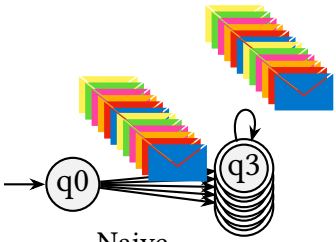


Naive

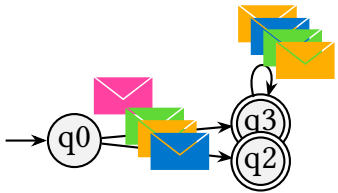
Naive answer: Just build DFAs
(accept the state space blowup!)

Better answer:

- Build a fragment of the naive, concrete automaton we call the *evidence automaton*



Naive

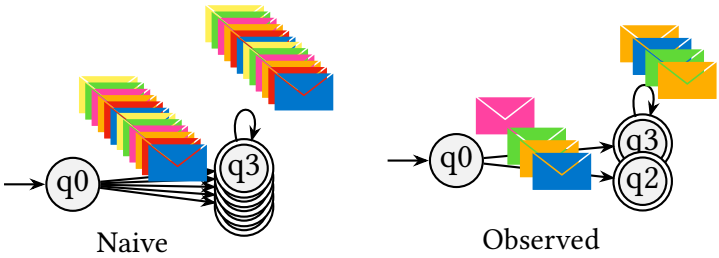


Observed

Naive answer: Just build DFAs
(accept the state space blowup!)

Better answer:

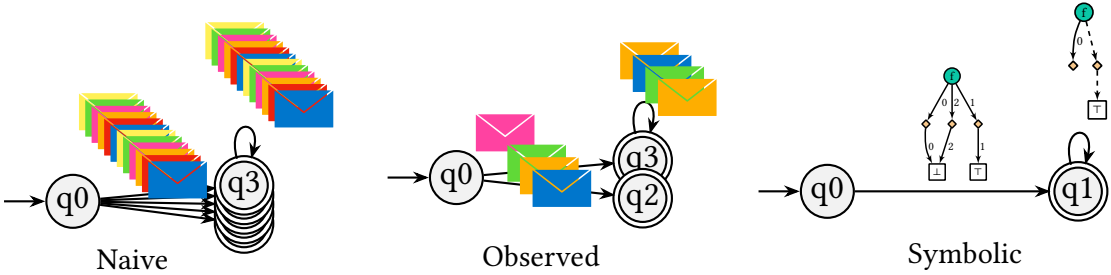
- Build a fragment of the naive, concrete automaton we call the *evidence automaton*



Naive answer: Just build DFAs
(accept the state space blowup!)

Better answer:

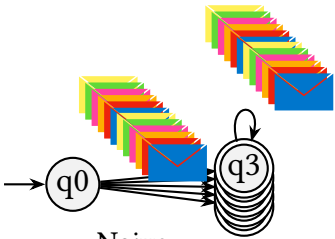
- Build a fragment of the naive, concrete automaton we call the *evidence automaton*
- Generalize it to a full symbolic automaton by generalizing the evidence on each transition



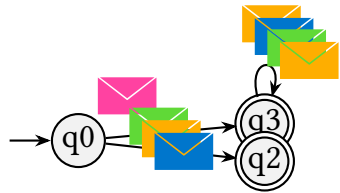
Naive answer: Just build DFAs
(accept the state space blowup!)

Better answer:

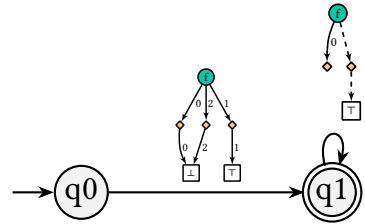
- Build a fragment of the naive, concrete automaton we call the *evidence automaton*
- Generalize it to a full symbolic automaton by generalizing the evidence on each transition



Naive



Observed



Symbolic

Naive answer: Just build DFAs
(accept the state space blowup!)

Better answer:

- Build a fragment of the naive, concrete automaton we call the *evidence automaton*
- Generalize it to a full symbolic automaton by generalizing the evidence on each transition



Key Contributions of NKL*:

1. We learn *symbolic* NetKAT automata
   - they can be used for **KATch** verification!

Key Contributions of NKL*:

1. We learn *symbolic* NetKAT automata
   - they can be used for **KATch** verification!

2. Termination proof via a **Myhill-Nerode theorem** for NetKAT

Key Contributions of NKL*:

1. We learn *symbolic* NetKAT automata
   - they can be used for **KATch** verification!

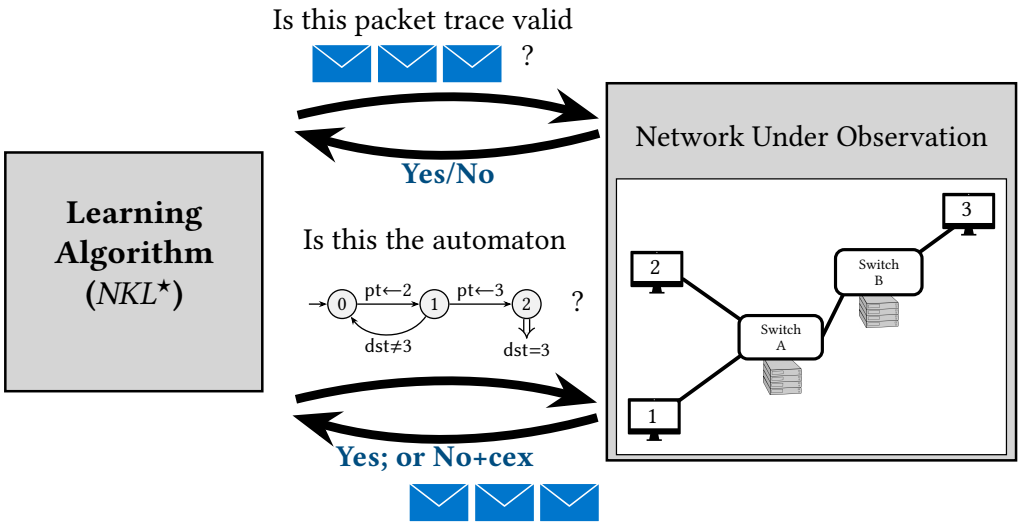
2. Termination proof via a **Myhill-Nerode theorem** for NetKAT

3. Prototype OCaml implementation



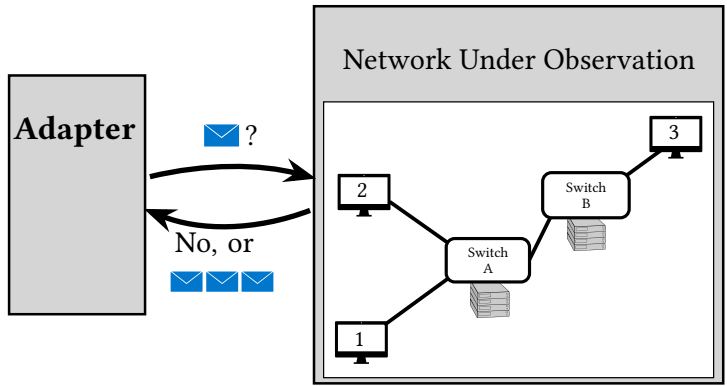
Ongoing Work in Automata-based Network Verification

Teacher's queries, revisited

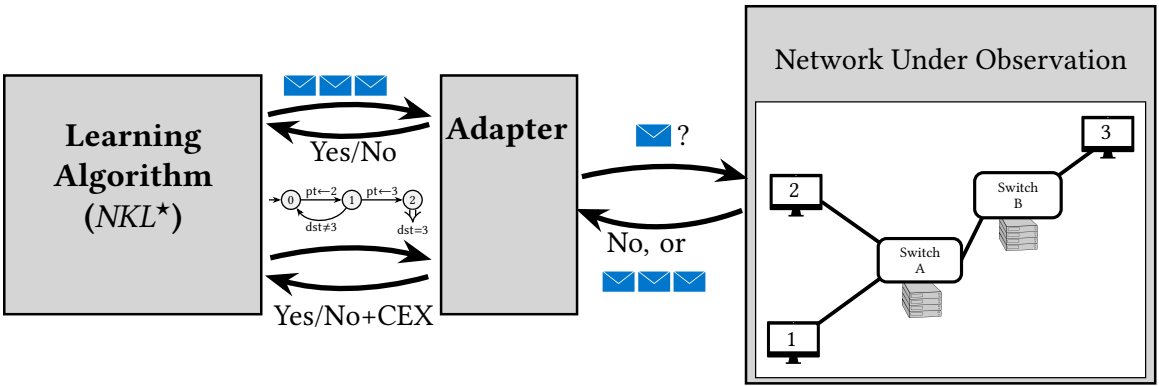


Teacher's queries, revisited

A more realistic membership query

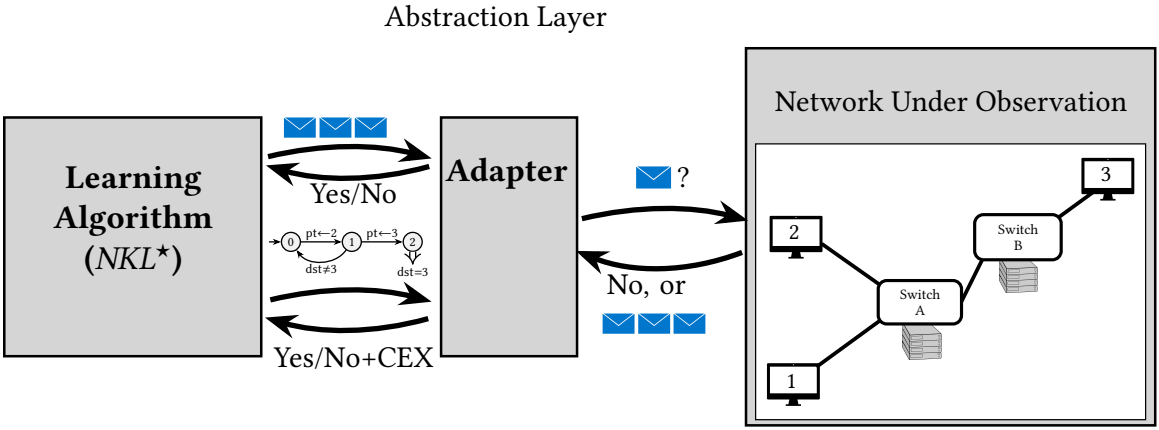


Teacher's queries, revisited



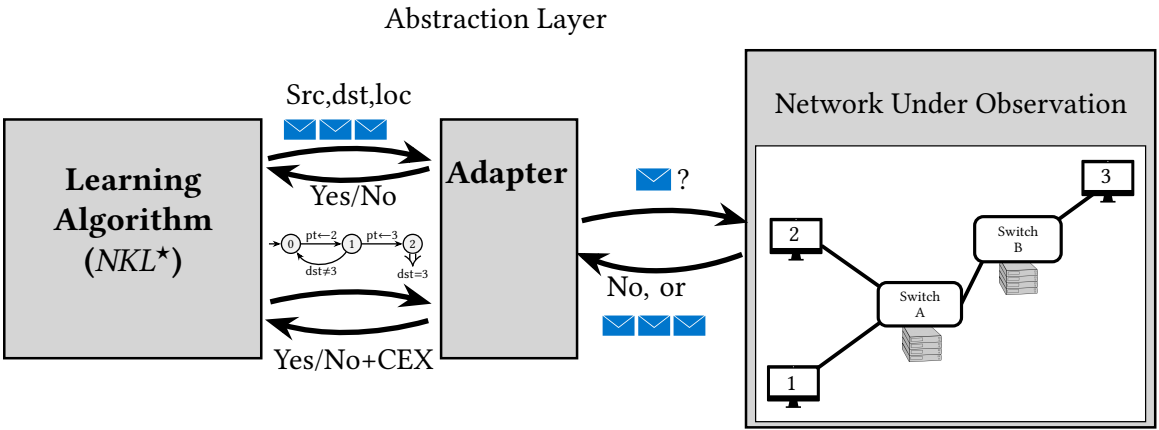
See e.g. Ferreira et al, 2021

Teacher's queries, revisited



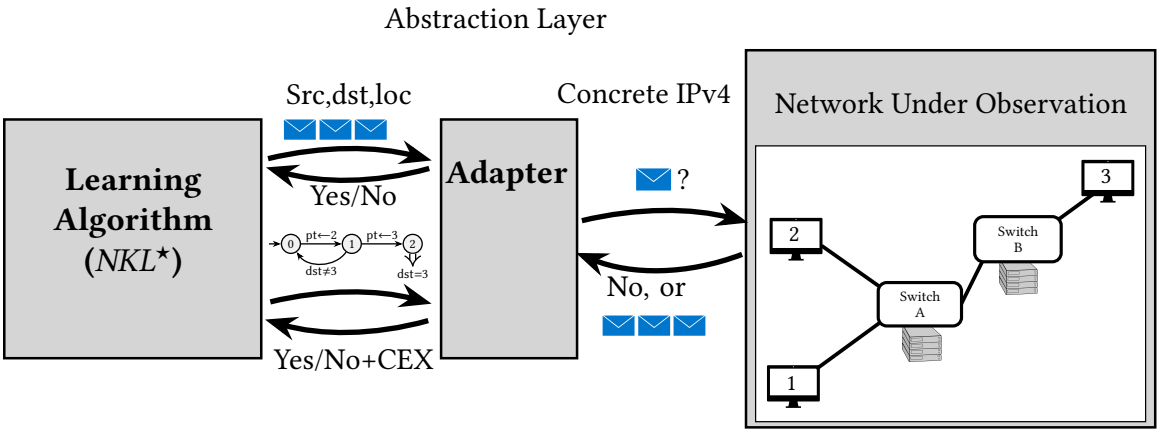
See e.g. Ferreira et al, 2021

Teacher's queries, revisited



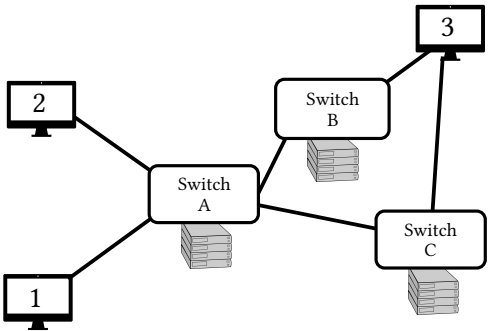
See e.g. Ferreira et al, 2021

Teacher's queries, revisited

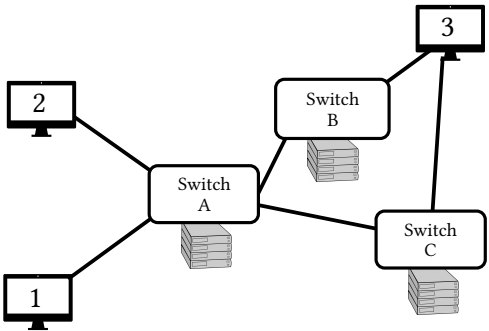


See e.g. Ferreira et al, 2021

Verification Revisited: Quantitative Properties



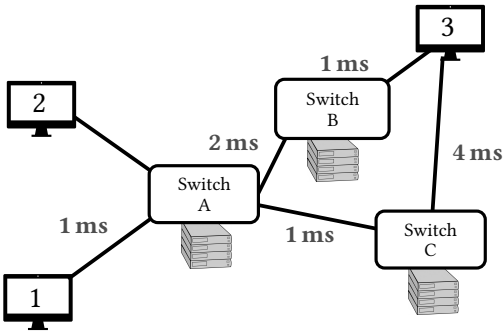
Verification Revisited: Quantitative Properties



Reachability (NetKAT):
Can host 1 reach host 3? ⇒ **Yes**

Verification Revisited: Quantitative Properties

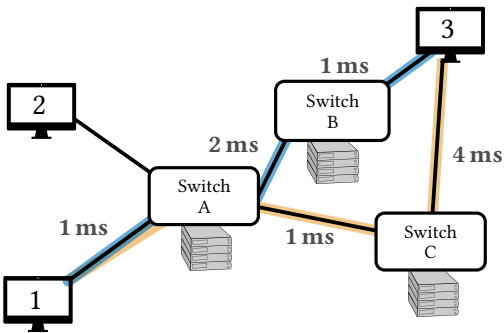
Link latencies



Reachability (NetKAT):
Can host 1 reach host 3? ⇒ **Yes**

Verification Revisited: Quantitative Properties

Link latencies



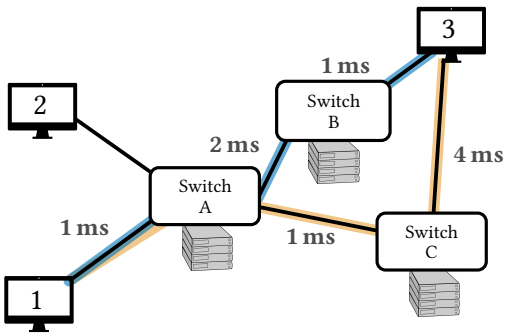
Reachability (NetKAT):
Can host 1 reach host 3? ⇒ **Yes**

Latency (weighted NetKAT):
How long from host 1 to host 3?

via B: 1 + 2 + 1 = 4 ms
via C: 1 + 1 + 4 = 6 ms

Verification Revisited: Quantitative Properties

Link latencies



Reachability (NetKAT):
Can host 1 reach host 3? ⇒ **Yes**

Latency (weighted NetKAT):
How long from host 1 to host 3?

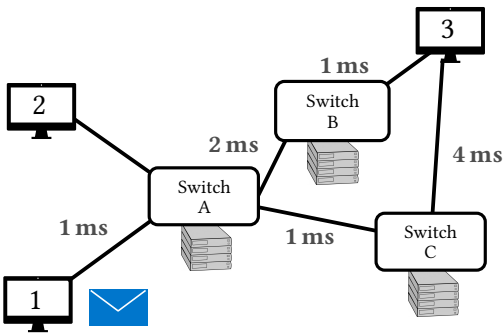
via B: 1 + 2 + 1 = 4 ms
via C: 1 + 1 + 4 = 6 ms

Link weights come from a semiring, e.g. Arctic, Tropical, Bottleneck, Reals, Viterbi, Boolean

Weighted NetKAT: Suárez Acevedo, Ferreira, Batz, Bøving, Foster, Silva (PLDI 2026)

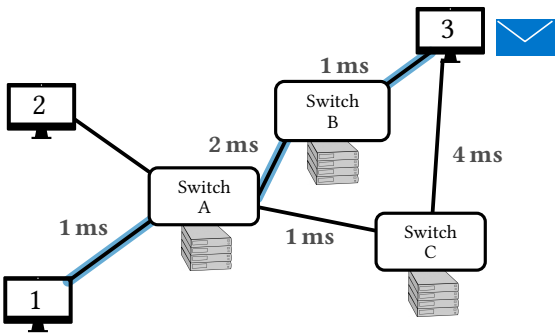
Learning *Weighted* NetKAT: What does each observation mean?

Send a probe from host 1 to host 3 and observe the result.



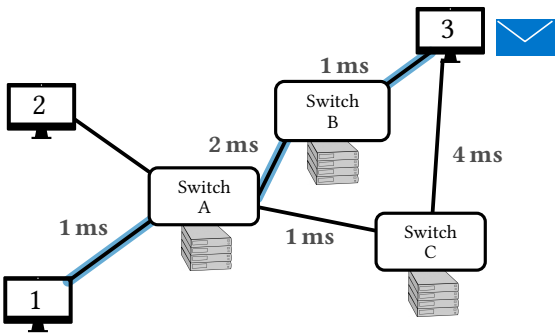
Learning *Weighted* NetKAT: What does each observation mean?

Send a probe from host 1 to host 3 and observe the result.



Learning *Weighted* NetKAT: What does each observation mean?

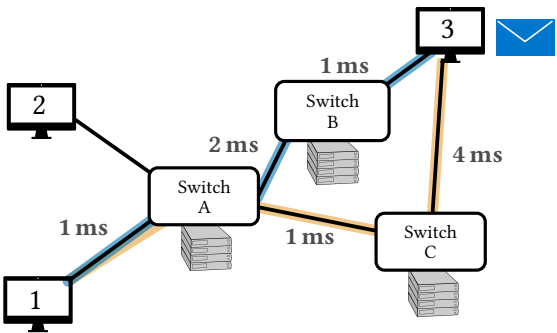
Send a probe from host 1 to host 3 and observe the result.



NKL*: it arrived
$\Rightarrow$ **this trace is a valid behavior**

Learning *Weighted* NetKAT: What does each observation mean?

Send a probe from host 1 to host 3 and observe the result.

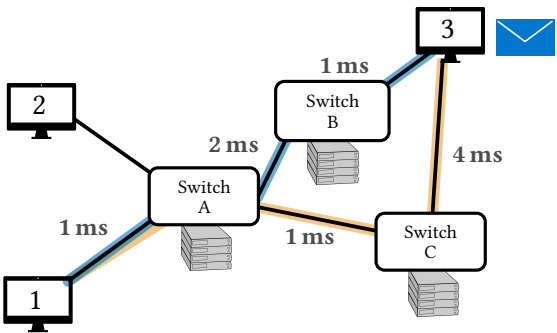


NKL*: it arrived
$\Rightarrow$ **this trace is a valid behavior**

Latency: it took 4 ms
$\Rightarrow$ worst case $\geq$ 4 ms: **a bound**
(via C: 6 ms, never seen)

Learning *Weighted* NetKAT: What does each observation mean?

Send a probe from host 1 to host 3 and observe the result.



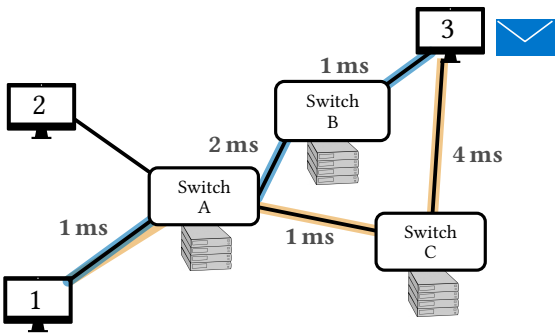
NKL*: it arrived
⇒ **this trace is a valid behavior**

Latency: it took 4 ms
⇒ worst case ≥ 4 ms: **a bound**
(via C: 6 ms, never seen)

Reliability: it arrived
⇒ **one sample** of a random outcome

Learning *Weighted* NetKAT: What does each observation mean?

Send a probe from host 1 to host 3 and observe the result.



NKL*: it arrived
⇒ **this trace is a valid behavior**

Latency: it took 4 ms
⇒ worst case ≥ 4 ms: **a bound**
(via C: 6 ms, never seen)

Reliability: it arrived
⇒ **one sample** of a random outcome

Single observations mean very different things in different semirings.

Learning & Verification with NetKAT

	Verify	**Learn**
Boolean	NetKAT (POPL14/15) ⇒ **KATch** (PLDI24)	**NKL*** (PLDI25)
Quantitative		

Learning & Verification with NetKAT

	Verify	Learn
Boolean	NetKAT (POPL14/15) ⇒ **KATch** (PLDI24)	**NKL*** (PLDI25)
Quantitative	wNetKAT (PLDI26) ⇒ **weighted SPPs** (POPL27)	

Learning & Verification with NetKAT

	Verify	Learn
Boolean	NetKAT (POPL14/15) ⇒ **KATch** (PLDI24)	**NKL⋆** (PLDI25)
Quantitative	wNetKAT (PLDI26) ⇒ **weighted SPPs** (POPL27)	? What does evidence mean?

Learning & Verification with NetKAT

	Verify	Learn
Boolean	NetKAT (POPL14/15) ⇒ **KATch** (PLDI24)	**NKL⋆** (PLDI25)
Quantitative	wNetKAT (PLDI26) ⇒ **weighted SPPs** (POPL27)	



Thomas Lu
lead author,
POPL27

Automata Learning for Network Verification

Mark Moeller + Jules Jacobs, Olivier Savary Belanger, David Darais, Cole Schlesinger, Steffen Smolka, Nate Foster, Alexandra Silva. *KATch: A Fast Symbolic Verifier for NetKAT.* PLDI 2024.

Mark Moeller, Tiago Ferreira, Thomas Lu, Nate Foster, Alexandra Silva. *Active Learning of Symbolic NetKAT Automata.* PLDI 2025.

Thomas Lu, Qiancheng Fu, Kevin Batz, Oliver Bøving, Tiago Ferreira, **Mark Moeller**, Nate Foster, Alexandra Silva. *A Fast Quantitative Analyzer for NetKAT.* To appear in POPL 2027.

mmoeller@holycross.edu · markmoeller.work